

지도교수: 이상준

2026.06.04

ROI 기반 MSA 선택적 전환 결정 기법

ROI Based MSA Selective Transition Decision Technique

저자: 소프트웨어공학과 4102410005 하지연





목차

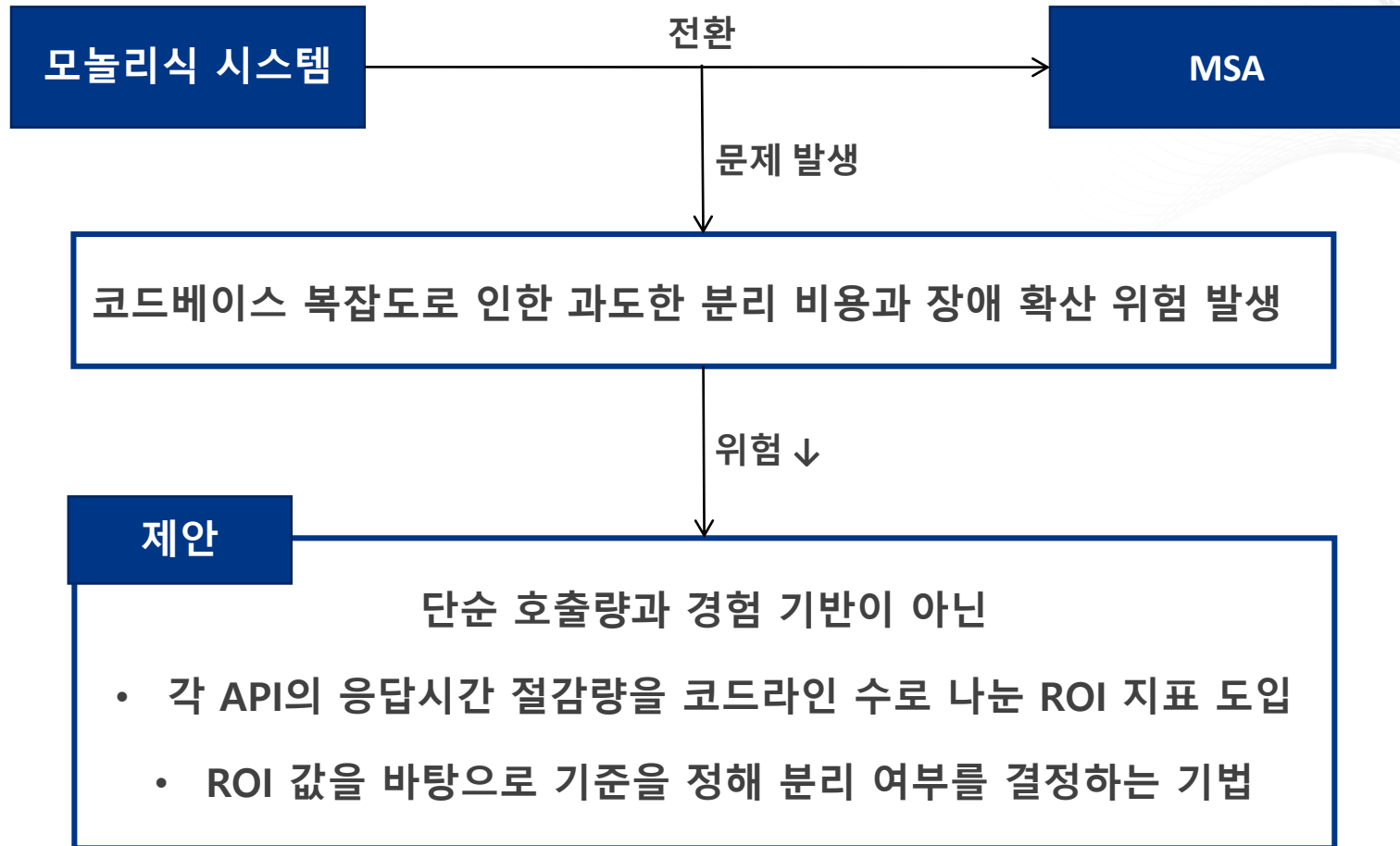
I 개요

II 지적사항 개선

III 실험 및 평가

IV 결론





실험 시나리오

구현

- 모놀리식 구조와 MSA 구조 구현
- Spring boot 기반으로, MSA는 마이크로 서비스로 분리

측정

- 주요 API를 대상으로 성능 측정 수행
- JMeter로 응답시간 10회 반복 측정 후 개발 코드라인 계산

분석

- 성능 개선도를 개발 비용으로 나눈 ROI 지표 도출 후 분석

계산 수식

- $(\text{DDD 응답시간} - \text{MSA 응답시간}) / (\text{MSA 코드라인} - \text{DDD 코드라인})$

예시) 모놀리식 평균 300ms, MSA 평균 150ms, 코드라인 수의 차이값 100줄인 경우 ROI
는 1.5(150ms/100줄)

지적사항 개선

LoC를 비용 지표로 사용한 이유

- LoC는 시스템의 규모와 관리 포인트를 명확하게 보여주는 프록시 지표
- 초기 모델로서 주관적 요소 대신, 물리적인 수정 범위를 나타내는 LoC를 비용의 최소 단위로 설정
- 현업 SI 구축 사업의 단가 산정 방식과 일치하는 실무적 접근

MSA 전환 시 LoC 증가 문제

- 절대적 LoC 증가량이 아닌 기능 분리 시 발생하는 구조적, 상대적 차이 식별에 집중
- 모든 기능에 공통으로 발생하는 오버헤드를 제외하고, 특정 기능을 위해 투입된 노력이 실제 성능으로 얼마나 돌아오는지 비교
- 기초 모델이기 때문에, 향후 연구를 통해 가중치 수용 가능

실험 및 평가

- API별 코드라인 수 (LOC) 비교

	API1&2	API3	API4	API5	전체
DDD	57	51	20	28	395 (LOC)
MSA	153	113	80	85	593 (LOC)

- Jmeter 10회 결과 평균

	API1(POST)	API2(GET)	API3(POST)	API4(PUT)	API5(GET)
DDD	530	7	463	353	39
MSA	2774	4	811	1661	8

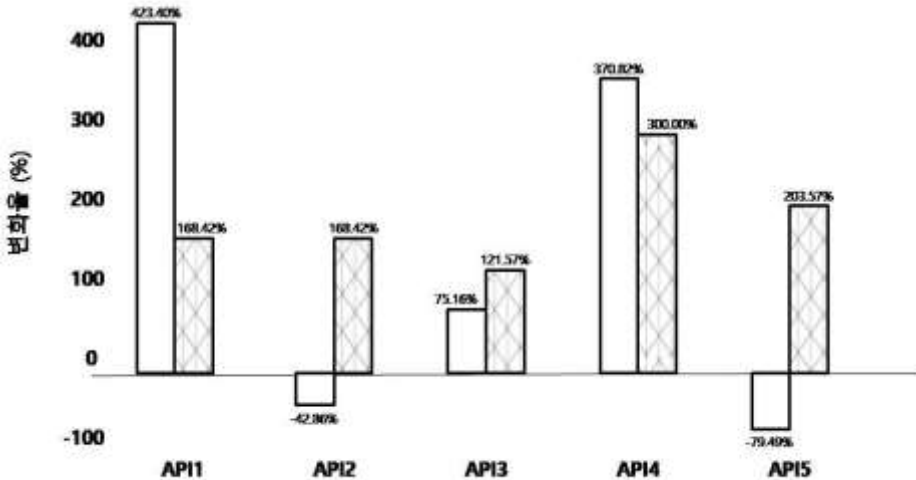
- ROI 기반 수식 적용한 결과값

API1	API2	API3	API4	API5
-23.37	0.03125	-5.61	-21.81	0.54

실험 및 평가

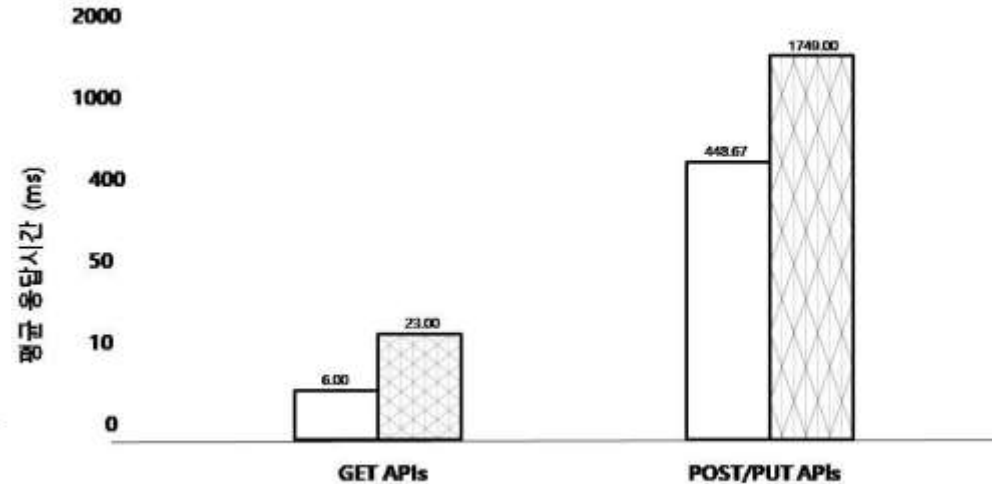
API별 성능 변화를 vs 코드 증가율

□ 성능변화율 □ 코드증가율



API별 유형별 평균 응답시간 비교

□ DDD □ MSA



- 조회 기반(GET) API2, API5는 성능 유지 및 개선 확인
- 쓰기 기반(POST/PUT) API1, API3, API4는 성능 저하 관찰
- Saga 패턴은 분산 트랜잭션 처리를 위한 단계별 응답 대기 및 보상 로직으로 인해 응답시간 급증
- 선택적 전환은 일관 전환 대비 개발 투입 공수(LoC) 약 36% 절감

실험 및 평가

- 서비스 간 HTTP 통신으로 인한 밀리초 단위 지연 시간이 쓰기 작업에서 치명적으로 작용
- 동일 수준의 코드 증가에도 API 유형에 따라 성능 변화율이 상반되게 나타남
- MSA 도입은 비즈니스 가치와 기술적 효율성을 고려한 하이브리드 설계의 영역
- ROI 기반 수식은 무분별한 아키텍처 전환에 따른 성능 위험을 사전에 차단하는 유효한 의사결정 지표

- 본 논문은 MSA 전환 시 ROI를 정량화하는 것을 목적으로 하며, 성능 지표와 개발 비용을 결합한 ROI 기반 선택적 마이그레이션 프레임워크를 제안하고 검증함
- 기존 마이크로서비스 연구는 확장성 등 정성적 지표에 치중되어 실제 전환 우선순위 결정에 어려움이 있었으나, 재무 ROI 개념을 차용한 정량적 효율 지표를 새롭게 정의하여 아키텍처 전환의 객관적 의사결정 기준을 마련함
- 제안 모델을 통한 실험 결과, 일괄 전환 대비 선택적 마이그레이션 시 개발 비용을 약 36% 절감하면서도 성능 이득을 극대화할 수 있음을 확인하여 모델의 실무적 유효성을 검증함
- 향후 비즈니스 임팩트와 트래픽 변수를 반영한 가중치산정 알고리즘을 추가하여 모델을 고도화하고, 다양한 도메인 환경에서 시스템의 최적화된 하이브리드 설계 방향을 확립할 필요가 있음

참고문헌

- [1] Yoon, H. et al., “Domain-Driven Design Representation of Monolith Candidate Decompositions Based on Entity Accesses,” Proc. of the Korean Institute of Information Scientists and Engineers, 2025.
- [2] Evans, E., “Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley Professional.”, 2003
- [3] Park, J. “마이크로서비스 아키텍처 기반 애플리케이션 프레임워크 연구,” JITA, vol.18, no.4, pp.309-318, 2024.
- [4] Kim, S.W. “마이크로서비스 아키텍처의 빛과 그림자,” Journal of Korea Contents Association, 2024.
- [5] Azure – <https://learn.microsoft.com/en-us/azure/architecture/patterns/saga>
- [6] AWS – <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/saga-choreography.html>
- [7] R. van Solingen, “Measuring the ROI of Software Process Improvement,” IEEE Software, 2004.
- [8] B. Farbey et al., “Evaluation in Software Engineering: ROI, but more than ROI,” 2001.
- [9] Davide Taibi, Kari Systä. “A Decomposition and Metric-Based Evaluation Framework for Microservices.” Proceedings of the 9th International Conference on Cloud Computing and Services Science, May 2019.
- [10] Yuyang Wei, Yijun Yu, Minxue Pan, and Tian Zhang. “A Feature Table approach to decomposing monolithic software systems into microservices.” Proceedings of the 12th Asia-Pacific Symposium on Internetware, May 2021.
- [11] Matt Watson. “Microservices ROI: A Comprehensive Cost-Benefit Analysis.” Fullscale, April 9, 2025.
- [12] Jakub Pepliński. “Monolith to Microservices – Own the Migration Process.” Alokai, June 2024.
- [13] Nuno Alexandre Vieira Santos. “A Complexity Metric for Microservices Architecture Migration.” IEEE International Conference on Software Architecture, March 2020.
- [14] Jonas Fritsch, Justus Bogner, Alfred Zimmermann, and Stefan Wagner. “Microservices Migration in Industry: Intentions, Strategies, and Challenges.” IEEE International Conference on Software Maintenance and Evolution, October 2019.



감사합니다.
